WFLOW-95 LAB: CMDB-Driven Host Tag Enrichment

> **Series:** WFLOW — Workflows and Alert Notifications | **Reference:** 95 — CMDB-Driven Host Tag Enrichment LAB | **Created:** June 2026 | **Last Updated:** 11/13/2026

Overview

A frequent ask: read host→application mappings from a CMDB and apply them as host tags in bulk. This hands-on LAB builds that end to end as a Dynatrace **Workflow** — two *Run JavaScript* tasks that enrich hosts from CMDB lookup tables and apply the tags via the OneAgent Remote Configuration Management API, safely (dry-run first) and idempotently (skip tags that already exist).

It is the capstone for **WFLOW-08 (JavaScript & HTTP Actions)** — it combines an SDK DQL query (WFLOW-08 §2), `fetch()` to a Dynatrace API with auth (§3), retry logic (§4) and `result()` / `withItems` data passing between tasks (§7) into one deployable workflow. Build it in the editor by following the steps, or import the YAML skeleton at the end.

> **Is this the right tool?** This LAB writes tags onto hosts through the classic remote configuration API. If the value can be derived from context the host already reports — host group, host name, an existing host tag — Latest Dynatrace has a simpler path: **Ingest enrichment configuration** (FAQ-02 § 3.3), a central rule with no host changes and no agent restart (OneAgent 1.343+). Use this LAB when the value exists **only in an external CMDB**, one value per host.

Table of Contents

1. What This LAB Builds

2. The Two Tag Surfaces
3. Safety Model
4. Build It in the Workflows Editor
5. Prefer to Import? (YAML Skeleton)
6. Adapting It
7. Next Steps
8. Bonus — Seed Sample Lookup Tables (testing only)

Prerequisites

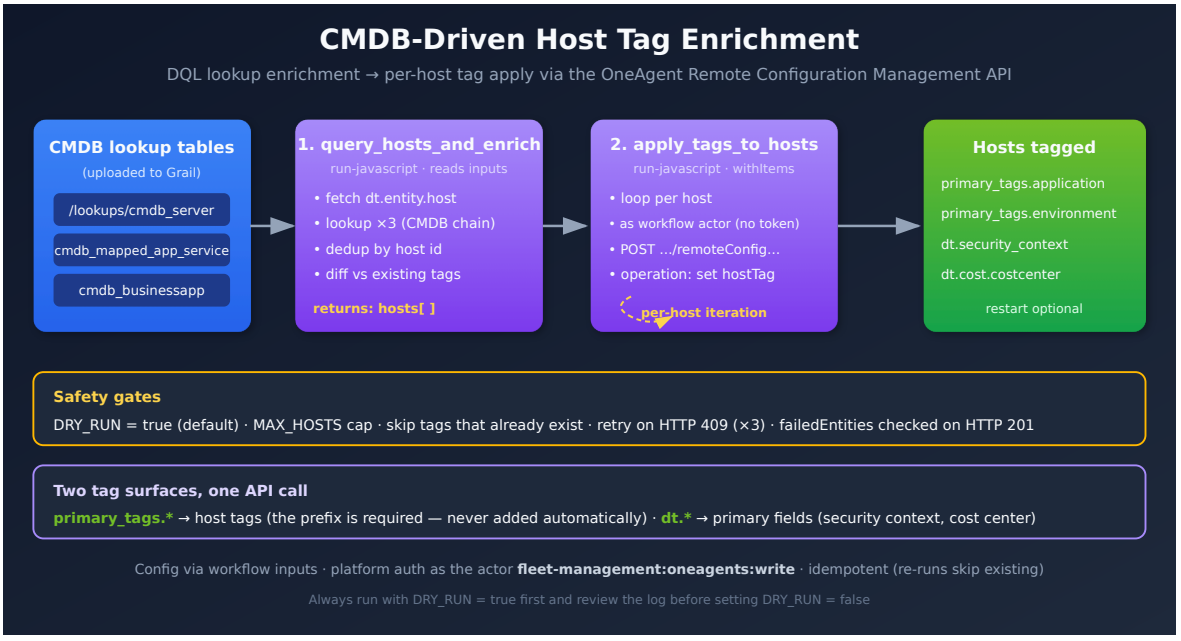
Requirement	Details
Dynatrace Environment	SaaS with Workflows (AutomationEngine) and Grail
Permissions	To build and run the workflow: `automation:workflows:write`, `automation:workflows:run`, `app-engine:functions:run`. **Workflow actor:** `storage:entities:read` (`dt.entity.host`), `storage:files:read` (the lookup tables) and `fleet-management:oneagents:write` (the remote configuration API, platform authentication — SaaS 1.343+), each also enabled in **Workflows > Settings > Authorization settings**. *Classic/hybrid alternative only:* a classic API token with `oneAgents.write` in the Credential Vault, and `environment-api:credentials:read` on the actor
Hosts	OneAgents in Full-Stack or Infrastructure Monitoring mode, connected to the environment. Remote configuration does not work with Operator-deployed or application-only OneAgents, or on Solaris
Data	Your CMDB exports uploaded as Grail lookup tables
Prior Knowledge	**WFLOW-08** (JavaScript & HTTP actions); DQL `lookup` / `load`

1. What This LAB Builds

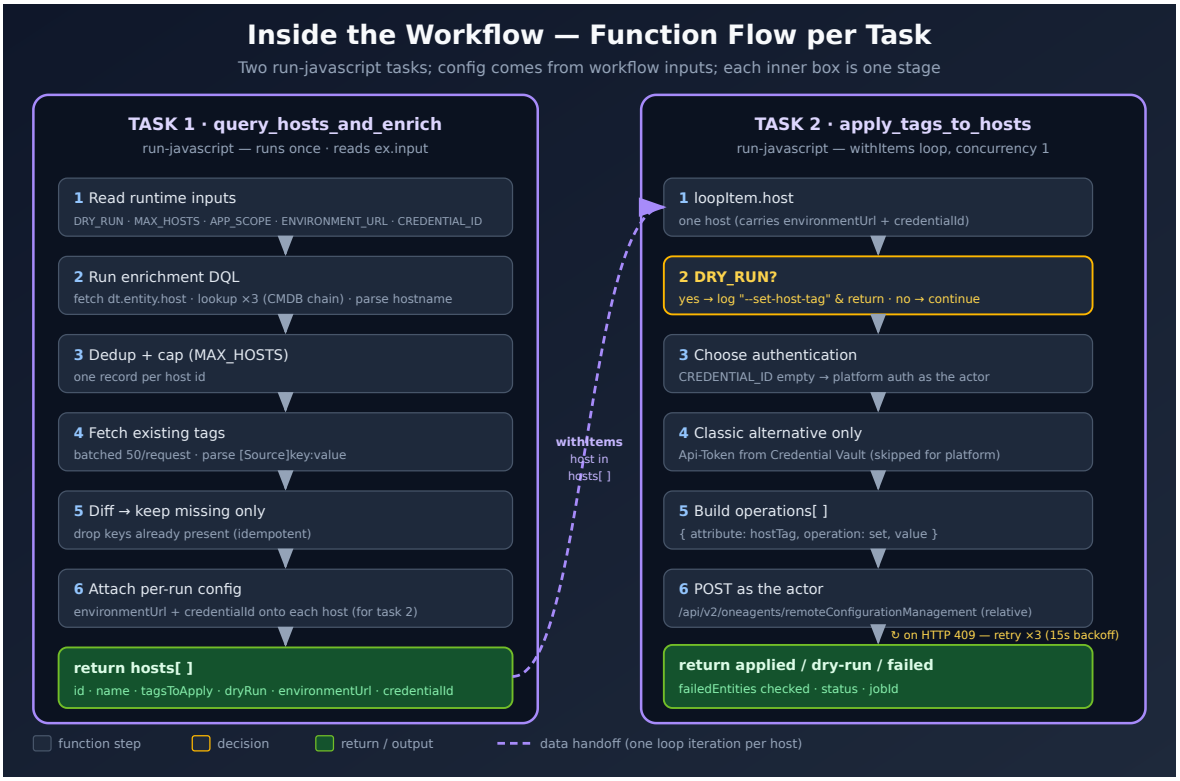
The workflow is two `run-javascript` tasks chained by a loop:

Task	Type	Role
`query_hosts_and_enrich`	`run-javascript`	Runs the DQL enrichment (host → three-table CMDB lookup chain), dedups by host id, fetches each host's current tags, and returns a `hosts[]` array containing **only the tags that are missing**.
`apply_tags_to_hosts`	`run-javascript` with a loop	For each host in `hosts[]`, POSTs a `set hostTag` operation to the **OneAgent Remote Configuration Management API** as the workflow actor (relative URL, no stored token) — respecting dry-run, retrying on HTTP 409 and checking `failedEntities`.

The write mechanism is `POST /api/v2/oneagents/remoteConfigurationManagement` with a `set hostTag` operation — the same `--set-host-tag` form you would run locally with `oneagentctl`, applied remotely across many hosts in one workflow.



The two tasks are chained: task 2 loops over `result('query_hosts_and_enrich').hosts` (one run per host) and only runs when task 1 found at least one host. The diagram below opens each task into its internal function flow.



2. The Two Tag Surfaces, One API Call

`TAG_MAPPING` maps an enrichment field to a host-tag key. Two different Dynatrace surfaces are written through the **same** `set hostTag` operation:

- `primary_tags.application` / `primary_tags.environment` become **primary tags**. The `primary_tags.` prefix is mandatory and is **never added for you** — a bare `application` key would create an ordinary tag, not a primary tag.
- `dt.security_context` and `dt.cost.costcenter` are **primary fields** (security context, cost-allocation), set through that same operation.

This split — and the primary-tags prefix rule — is covered in depth in **FAQ-02 (Tagging: Sources, Standards & Strategy)**. Tune `TAG_MAPPING` to the fields and tag keys your environment actually uses.

3. Safety Model

The workflow defaults to **safe**: a first run changes nothing. Read the dry-run log, confirm the proposed tags, then flip one flag.

Gate	What it does
`DRY_RUN = true` *(default)*	Task 2 only logs the `--set-host-tag` it *would* run — no write. Set to `false` to apply.
`MAX_HOSTS`	Caps hosts processed per run (default `50`; `0` = unlimited).
Skip-existing	Task 1 reads current tags and drops any key already present — re-runs are idempotent.
409 retry	HTTP 409 means *another* remote configuration job is running in the environment — any bulk action, by a person or another workflow, not only this one. Task 2 retries up to `MAX_RETRIES` (3) with a 15s backoff.
`failedEntities` check	A 201 response returns only after every OneAgent in the job is processed, and lists agents that failed (`CONNECTION_FAILURE`, `TIMEOUT`) in `failedEntities`. Task 2 reports those hosts as `failed`, not `applied`.
No stored secret	Task 2 calls the API with a **relative URL**, so the runtime authenticates it as the workflow actor — no token in the vault or the workflow.
Config validation	In the classic-token alternative, task 2 throws *before any live write* if `ENVIRONMENT_URL` / `CREDENTIAL_ID` still hold placeholders.
Loop concurrency `1`	Hosts are tagged one at a time (serialized loop) to avoid colliding jobs and API pressure.

> **`RESTART_ONEAGENT = false` means "accepted", not "applied".** The API reference states that *"By default OneAgents will be restarted when network zone, host group, host tags or host properties are reconfigured - the restart is required to apply the changes."* The LAB defaults to no restart so that a first live run cannot bounce agents unexpectedly — but with it off, a successful job (HTTP 2xx, a `jobId`) does not mean the tags are on the data yet; they apply when each host's OneAgent next restarts. Either schedule a restart window and set `RESTART_ONEAGENT = true`, or verify on data ingested after the hosts' next restart. Removals are slower still: *"Removing host properties and tags may require up to seven hours to take effect."*
>
> **Sources:** POST a configuration job (DT docs) — *"409 - Other remote configuration management job is currently being executed"* and *"The response is not sent to the client until all OneAgents defined in*

> *the payload are processed."*; Remote configuration management (DT docs) — *"You can't start another bulk action until the current one is finished."*

4. Build It in the Workflows Editor (Step by Step)

Two *Run JavaScript* tasks wired with a loop. **All configuration lives in workflow inputs** (runtime variables) — task 1 reads them with `ex.input` and passes the values task 2 needs onto each host object, so the loop task needs no input wiring of its own.

> **UI labels vary between Workflows app versions.** Where you define workflow inputs and the loop differs by version; the authoritative shape is the exported YAML (see the import skeleton, which includes the `input:` block verbatim). The workflow is adapted from a working deployment that used a classic token, with its tenant URL and names genericized, rather than from a Dynatrace-published example — keep `DRY_RUN = true` for the first run in your tenant.

Step 0 — Prerequisites (once):

- **Upload your CMDB exports as Grail lookup tables** at the paths the query references (`/lookups/cmdb_server` , `/lookups/cmdb_mapped_app_service` , `/lookups/cmdb_businessapp`); adjust names/ `fields` to your schema. *(If they're missing, task 1's preflight stops the run cleanly with a "Missing CMDB lookup tables" message — no raw `UNKNOWN_TABULAR_FILE` error.)*
- **Give the workflow's actor the permissions** in the Prerequisites — above all `fleet-management:oneagents:write` — and enable them in **Workflows > Settings > Authorization settings**. A permission missing from either place makes the task fail with 403 Forbidden. A service user is the recommended actor for a production workflow (WFLOW-09 §4).
- **Classic or hybrid environments that keep a classic token instead:** create a classic API token with **`oneAgents.write`** , store it in the **Credential Vault** with **AppEngine** scope and **Allow access without app context** turned on, give the workflow actor access to the credential and `environment-api:credentials:read` , and set the `ENVIRONMENT_URL` and `CREDENTIAL_ID` inputs. Latest environments have no classic tokens, so this path does not exist there.

> **Platform authentication (SaaS 1.343+).** Since SaaS 1.343, the OneAgent fleet-management endpoints accept platform authentication, and the API reference lists *"Platform Token / OAuth: Required scope: fleet-management:oneagents:write"*. Task 2 therefore calls the endpoint with a **relative URL**: the Run JavaScript runtime attaches the authentication for the actor, so no token is stored anywhere. This path follows the documentation but has **not been tested live** for this endpoint — keep `DRY_RUN = true`, then try one host with `MAX_HOSTS = 1` before a full run. If your tenant is not yet on 1.343, use the classic-token alternative.
>
> **Sources:**
>
> - Run JavaScript action (DT docs) — *"Use relative paths, for example /api/v2/entities , to call Dynatrace APIs. The runtime automatically attaches the required authentication headers. Don't set a custom Authorization header on relative URL requests."* and *"The AppEngine scope is selected. Allow access without app context is turned on. The workflow actor has access to the credential."*
> - POST a configuration job (DT docs) — *"Platform Token / OAuth: Required scope: fleet-management:oneagents:write"*
> - Sprint 343 release notes (DT docs) — *"We recommend platform tokens over classic API tokens for new integrations."*
> - Upgrade from classic access tokens (DT docs) — (latest environment) *"Platform tokens only. Classic access tokens don't exist, and v2/apiTokens isn't available."*
> - Workflow security (DT docs) — *"If the required permission for a workflow task is missing, an attempt to execute this task results in a 403 Forbidden error."*

Step 1 — Create the workflow and define its inputs. New workflow (leave the on-demand trigger). Define these **workflow inputs** — they become the parameters you can override in the Run dialog (easiest to set via the YAML editor — see the import skeleton):

Input	Default	Purpose
`DRY_RUN`	`true`	preview only; flip to `false` to apply
`MAX_HOSTS`	`50`	cap per run; `0` = unlimited
`APP_SCOPE`	`''`	limit to one business app; `''` = all
`EXCLUDE_PRODUCTION`	`false`	skip `environment == production`
`RESTART_ONEAGENT`	`false`	restart OneAgent after tagging

Input	Default	Purpose
`ENVIRONMENT_URL`	`''`	classic-token alternative only: tenant API base (no trailing slash)
`CREDENTIAL_ID`	`''`	classic-token alternative only: vault entry holding the `oneAgents.write` token. Empty = platform auth

Step 2 — Add the first JavaScript block. Click **+** under the trigger → **Run JavaScript**. Rename it `query_hosts_and_enrich`.

Step 3 — Fill it with this code. It preflights the lookup tables (so a missing upload fails clean), then enriches and diffs:

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';
import { execution } from '@dynatrace-sdk/automation-utils';

export default async function ({ executionId }) {
  // ---- Runtime inputs (workflow `input` block; override per run) ----
  const ex = await execution(executionId);
  const inp = ex.input || {};
  const MAX_HOSTS = Number(inp.MAX_HOSTS ?? 50);
  const DRY_RUN = (inp.DRY_RUN === false || inp.DRY_RUN === 'false') ? false :
true; // default true
  const RESTART_ONEAGENT = (inp.RESTART_ONEAGENT === true ||
inp.RESTART_ONEAGENT === 'true');
  // APP_SCOPE is interpolated into DQL below — allow only a plain identifier
(no quotes, no pipes).
  const APP_SCOPE = (inp.APP_SCOPE ?? '').toString().trim().toLowerCase();
  if (APP_SCOPE &amp;&amp; !/^[a-z0-9._-]+$/.test(APP_SCOPE)) throw new
Error(`APP_SCOPE '${APP_SCOPE}' must match [a-z0-9._-]+`);
  const EXCLUDE_PRODUCTION = (inp.EXCLUDE_PRODUCTION === true ||
inp.EXCLUDE_PRODUCTION === 'true');
  // Optional — only for the classic-token alternative (task 2). Leave both
empty for platform auth.
  const ENVIRONMENT_URL = (inp.ENVIRONMENT_URL ??
'').toString().replace(/\/+$/, '');
  const CREDENTIAL_ID = (inp.CREDENTIAL_ID ?? '').toString();

  const TAG_MAPPING = {
    'application': 'primary_tags.application',
    'environment': 'primary_tags.environment',
    'dt.security_context': 'dt.security_context',
    'dt.cost.costcenter': 'dt.cost.costcenter',
  };

  console.log('=== Host Tag Enrichment (runtime inputs) ===');
  console.log(`DRY_RUN=${DRY_RUN} | MAX_HOSTS=${MAX_HOSTS} |
APP_SCOPE='${APP_SCOPE}' | EXCLUDE_PRODUCTION=${EXCLUDE_PRODUCTION} |
RESTART_ONEAGENT=${RESTART_ONEAGENT}`);

  // ---- Preflight: confirm the CMDB lookup tables exist before querying
them, ----
  // ---- so a missing upload stops cleanly instead of throwing
UNKNOWN_TABULAR_FILE. ----
  const LOOKUP_TABLES = ['/lookups/cmdb_server',
'/lookups/cmdb_mapped_app_service', '/lookups/cmdb_businessapp'];
  const missingTables = await checkLookupTables(LOOKUP_TABLES);
  if (missingTables.length &gt; 0) {
    console.log(`Missing CMDB lookup tables: ${missingTables.join(', ')}`);
    console.log('Upload them first (Step 0), then re-run. Stopping cleanly -
```

```
no hosts processed.');
    return { hosts: [], totalFound: 0, toProcess: 0, dryRun: DRY_RUN,
missingLookupTables: missingTables };
  }

  let enrichmentQuery = `fetch dt.entity.host
| fields id, entity.name, monitoringMode, paasVendorType,
isMonitoringCandidate, awsNameTag
| filter isMonitoringCandidate == false
// Remote configuration management does not work with Operator-deployed or
application-only
// OneAgents. PaaS hosts (paasVendorType set, e.g. Kubernetes) are excluded as
not eligible —
// the label says nothing about how they are billed or monitored.
| fieldsAdd rcm_eligibility = if(isNotNull(paasVendorType),
"NOT_RCM_ELIGIBLE", else: "ELIGIBLE"),
            provider = if(isNotNull(awsNameTag), "aws", else: "datacenter")
| filterOut rcm_eligibility == "NOT_RCM_ELIGIBLE"
| fieldsRemove paasVendorType, awsNameTag, isMonitoringCandidate
| parse entity.name, """LD:hostname ('.' LD:domain)? EOS"""
| lookup [load "/lookups/cmdb_server"
          | fields cmdb_ci_key, name, dv_operational_status, environment,
busapp_cmdb_ci_key, apps_cmdb_ci_key], sourceField:hostname, lookupField:name,
prefix:"server."
| lookup [load "/lookups/cmdb_mapped_app_service"
          | fields environment, managed_by_email, cmdb_ci_key, busapp_name],
sourceField:server.apps_cmdb_ci_key, lookupField:cmdb_ci_key,
prefix:"mapped_app."
| lookup [load "/lookups/cmdb_businessapp"
          | fields short_name, owned_by, cmdb_ci_key],
sourceField:server.busapp_cmdb_ci_key, lookupField:cmdb_ci_key,
prefix:"bus_app."
| fields id, entity.name, application = lower(bus_app.short_name), environment
= lower(mapped_app.environment), dt.cost.costcenter =
lower(bus_app.short_name), dt.security_context = lower(bus_app.short_name)
| filter isNotNull(application) OR isNotNull(environment)`;

  if (APP_SCOPE) enrichmentQuery += `\n| filter application ==
"${APP_SCOPE}"`; // validated above
  if (EXCLUDE_PRODUCTION) enrichmentQuery += `\n| filterOut environment ==
"production"`;

  const enriched = await executeDqlQuery(enrichmentQuery);
  console.log(`Found ${enriched.length} enriched host records`);

  const hostMap = new Map();
  enriched.forEach(record =&gt; {
    if (!hostMap.has(record.id)) {
```

```
      const hostData = { id: record.id, name: record['entity.name'],
application: record.application };
      Object.keys(TAG_MAPPING).forEach(f =&gt; { hostData[f] =
getNestedField(record, f); });
      hostMap.set(record.id, hostData);
    }
  });
  const uniqueHosts = Array.from(hostMap.values());
  const hostsToProcess = MAX_HOSTS &gt; 0 ? uniqueHosts.slice(0, MAX_HOSTS) :
uniqueHosts;
  console.log(`${uniqueHosts.length} unique hosts; processing
${hostsToProcess.length}`);
  if (hostsToProcess.length === 0) return { hosts: [], totalFound: 0,
toProcess: 0, dryRun: DRY_RUN };

  const BATCH = 50;
  const existing = new Map();
  for (let i = 0; i &lt; hostsToProcess.length; i += BATCH) {
    const batch = hostsToProcess.slice(i, i + BATCH);
    const filterParts = batch.map(h =&gt; `id == "${h.id}"`).join(' or ');
    const recs = await executeDqlQuery(`fetch dt.entity.host
| fieldsAdd tags
| filter ${filterParts}
| fields id, tags`);
    recs.forEach(record =&gt; {
      const allTags = (record.tags || []).filter(t =&gt; typeof t ===
'string');
      const keys = allTags.map(t =&gt; {
        let s = t; const b = t.lastIndexOf(']'); if (b !== -1) s =
t.substring(b + 1);
        const c = s.indexOf(':'); return c === -1 ? null : s.substring(0, c);
      }).filter(k =&gt; k &amp;&amp; Object.values(TAG_MAPPING).includes(k));
      existing.set(record.id, new Set(keys));
    });
  }

  const hostsWithTags = [];
  let skipped = 0;
  hostsToProcess.forEach(host =&gt; {
    const have = existing.get(host.id) || new Set();
    const tagsToApply = [];
    Object.entries(TAG_MAPPING).forEach(([f, tagKey]) =&gt; {
      const value = host[f];
      if (!value || String(value).trim().length === 0) return;
      if (have.has(tagKey)) { skipped++; return; }
      const clean = sanitizeTagValue(tagKey, value);
      if (clean) tagsToApply.push({ key: tagKey, value: clean });
    });
```

```
    if (tagsToApply.length &gt; 0) {
      hostsWithTags.push({ id: host.id, name: host.name, application:
host.application, tagsToApply,
        dryRun: DRY_RUN, restartOneAgent: RESTART_ONEAGENT, environmentUrl:
ENVIRONMENT_URL, credentialId: CREDENTIAL_ID });
    }
  });

  console.log(DRY_RUN ? 'DRY RUN - no changes' : 'LIVE - will apply tags');
  hostsWithTags.forEach((h, i) =&gt; { console.log(`  ${i + 1}. ${h.name}
(${h.id})`); h.tagsToApply.forEach(t =&gt; console.log(`     +
${t.key}=${t.value}`)); });
  console.log(`Hosts to tag: ${hostsWithTags.length} | tags skipped (exist):
${skipped}`);

  return { hosts: hostsWithTags, totalFound: uniqueHosts.length, toProcess:
hostsWithTags.length, totalSkipped: skipped, dryRun: DRY_RUN };
}

// Preflight helper: returns the subset of lookup-table paths that do NOT
exist.
async function checkLookupTables(paths) {
  const missing = [];
  for (const path of paths) {
    try {
      await executeDqlQuery(`load "${path}" | limit 1`);
    } catch (e) {
      if (e.message &amp;&amp; e.message.includes('UNKNOWN_TABULAR_FILE'))
missing.push(path);
      else throw e; // a real/unexpected error - surface it
    }
  }
  return missing;
}

function getNestedField(record, fieldName) {
  if (record[fieldName] !== undefined) return record[fieldName];
  return fieldName.split('.').reduce((v, p) =&gt; (v == null ? undefined :
v[p]), record);
}
// A host tag may not contain whitespace or '=' in its value, and `key=value`
may be at most 256 characters.
function sanitizeTagValue(key, value) { return String(value).replace(/[\s=]/g,
'').slice(0, Math.max(0, 256 - key.length - 1)); }
async function executeDqlQuery(query) {
  // Keep the token from queryExecute: poll responses do not carry one of
their own.
  const started = await queryExecutionClient.queryExecute({ body: { query,
```

```
requestTimeoutMilliseconds: 60000, maxResultRecords: 10000 } });
  let r = started;
  while (r &amp;amp;&amp;amp; (r.state === 'NOT_STARTED' || r.state === 'RUNNING')) {
    r = await queryExecutionClient.queryPoll({ requestToken:
started.requestToken, requestTimeoutMilliseconds: 30000 });
  }
  if (!r || r.state !== 'SUCCEEDED') throw new Error(`DQL failed: ${r?.state
?? 'no response'}`);
  return r.result.records;   // capped at maxResultRecords (10,000) - the
query API has no result paging
}
```

Step 4 — Customize task 1. Adjust `TAG_MAPPING`, the three `lookup [load "/lookups/…"]` tables and their `| fields …` lists, and the final `| fields … = lower(…)` mapping to match your CMDB. **If you rename the lookup tables, update both the `LOOKUP_TABLES` preflight list and the `lookup [load …]` calls.** Run scope and behavior (`DRY_RUN`, `MAX_HOSTS`, etc.) come from the **inputs**, not code constants.

Step 5 — Add the second JavaScript block. Click **+** beneath task 1 → **Run JavaScript**. Rename it `apply_tags_to_hosts` (placing it beneath task 1 makes task 1 its predecessor).

Step 6 — Fill it with this code:

```
import { actionExecution } from '@dynatrace-sdk/automation-utils';
import { credentialVaultClient } from '@dynatrace-sdk/client-classic-
environment-v2';

const MAX_RETRIES = 3;
const RETRY_DELAY_MS = 15000; // 15s between retries on 409
const API_PATH = '/api/v2/oneagents/remoteConfigurationManagement';

export default async function ({ actionExecutionId }) {
  const actionExe = await actionExecution(actionExecutionId);
  const host = actionExe.loopItem.host;
  const ENVIRONMENT_URL = host.environmentUrl;   // from task 1 (workflow
input) — classic alternative only
  const CREDENTIAL_ID = host.credentialId;       // from task 1 (workflow
input) — classic alternative only

  console.log(`--- ${host.name} (${host.id}) - ${host.tagsToApply.length}
tag(s) ---`);

  if (host.dryRun) {
    host.tagsToApply.forEach(t =&gt; console.log(`  [DRY RUN] --set-host-
tag=${t.key}=${t.value}`));
    return { host: host.name, hostId: host.id, status: 'dry-run', tagCount:
host.tagsToApply.length };
  }

  // Default: platform authentication. A relative URL is called as the
workflow actor — the runtime
  // attaches the authentication header; never set Authorization yourself. The
actor needs
  // fleet-management:oneagents:write.
  let url = `${API_PATH}?restart=${host.restartOneAgent}`;
  const headers = { 'Content-Type': 'application/json; charset=utf-8' };

  // Classic/hybrid alternative: a classic API token (oneAgents.write) read
from the Credential Vault.
  if (CREDENTIAL_ID) {
    if (!ENVIRONMENT_URL || ENVIRONMENT_URL.includes('&lt;')) throw new
Error('ENVIRONMENT_URL input is not set.');
    if (CREDENTIAL_ID.includes('X')) throw new Error('CREDENTIAL_ID input
still holds the placeholder.');
    const cred = await credentialVaultClient.getCredentialsDetails({ id:
CREDENTIAL_ID });
    if (!cred.token) throw new Error('Credential has no token value.');
    url = `${ENVIRONMENT_URL}${url}`;
    headers['Authorization'] = `Api-Token ${cred.token}`;
  }
```

```
  const operations = host.tagsToApply.map(tag =&gt; ({ attribute: 'hostTag',
operation: 'set', value: `${tag.key}=${tag.value}` }));

  for (let attempt = 1; attempt &lt;= MAX_RETRIES; attempt++) {
    try {
      const response = await fetch(url, {
        method: 'POST',
        headers,
        body: JSON.stringify({ entities: [host.id], operations })
      });
      // 409: another remote configuration job is running in the environment —
wait and retry.
      if (response.status === 409 &amp;&amp; attempt &lt; MAX_RETRIES) { await
new Promise(s =&gt; setTimeout(s, RETRY_DELAY_MS)); continue; }
      if (!response.ok) throw new Error(`HTTP ${response.status}: ${await
response.text()}`);
      const data = await response.json();
      // 201 means the job ran — not that every agent accepted it. Check
failedEntities.
      const failed = data.failedEntities || [];
      if (failed.length &gt; 0) {
        const reason = `${failed[0].failureReason}: ${failed[0].failureMessage
|| ''}`;
        console.error(`  FAILED on agent (job ${data.id}): ${reason}`);
        return { host: host.name, hostId: host.id, status: 'failed', error:
reason, jobId: data.id, attempt };
      }
      console.log(`  SUCCESS (attempt ${attempt}) jobId=${data.id || 'N/A'}`);
      return { host: host.name, hostId: host.id, status: 'applied',
tagsApplied: host.tagsToApply.length, jobId: data.id || 'N/A', attempt };
    } catch (error) {
      if (attempt &lt; MAX_RETRIES &amp;&amp; error.message &amp;&amp;
error.message.includes('409')) { await new Promise(s =&gt; setTimeout(s,
RETRY_DELAY_MS)); continue; }
      console.error(`  FAILED (attempt ${attempt}): ${error.message}`);
      return { host: host.name, hostId: host.id, status: 'failed', error:
error.message, attempt };
    }
  }
  return { host: host.name, hostId: host.id, status: 'failed', error: 'Max
retries exceeded' };
}
```

Step 7 — Customize task 2. Normally nothing. With `CREDENTIAL_ID` empty, task 2 uses platform authentication as the actor. Only in the classic-token alternative do

`ENVIRONMENT_URL` and `CREDENTIAL_ID` matter; they arrive from task 1 on each host object.

Step 8 — Make task 2 loop over the hosts. Enable the loop (Options → Loop / "Run task for each item"):

- **Items:** `{{ result('query_hosts_and_enrich').hosts }}`
- **Loop item variable name:** `host` — must be `host` so `actionExe.loopItem.host` resolves.
- **Concurrency:** `1` (serialize the writes).

Run condition (Conditions → Custom): `{{ result('query_hosts_and_enrich').hosts | length > 0 }}`

Step 9 — Run with inputs. Save → Run. Leave `DRY_RUN = true` (and set `ENVIRONMENT_URL` / `CREDENTIAL_ID` only for the classic-token alternative). Review the task-2 log's `--set-host-tag` lines, then re-run with `DRY_RUN = false` to apply. Add a schedule trigger (WFLOW-02) to keep newly onboarded hosts tagged; skip-existing keeps repeat runs cheap and idempotent.

5. Prefer to Import? (YAML Skeleton)

Build the envelope below — it includes the `input:` block and the loop/condition wiring — paste the **Step 3** and **Step 6** scripts into the `script:` blocks, and import it from the Workflows app.

```
metadata:
  version: '1'
  dependencies:
    apps:
      - id: dynatrace.automations
        version: ^1.3074.2
workflow:
  title: Host Tag Enrichment from Lookup Table
  schemaVersion: 4
  trigger: {}
  type: STANDARD
  input:
    DRY_RUN: true
    APP_SCOPE: ''
    MAX_HOSTS: 50
    ENVIRONMENT_URL: ''   # classic-token alternative only
    CREDENTIAL_ID: ''     # classic-token alternative only; empty = platform
auth
    RESTART_ONEAGENT: false
    EXCLUDE_PRODUCTION: false
  tasks:
    query_hosts_and_enrich:
      name: query_hosts_and_enrich
      action: dynatrace.automations:run-javascript
      predecessors: []
      input:
        script: |
          # paste the Step 3 script (query_hosts_and_enrich) here
    apply_tags_to_hosts:
      name: apply_tags_to_hosts
      action: dynatrace.automations:run-javascript
      predecessors:
        - query_hosts_and_enrich
      conditions:
        custom: '{{ result(''query_hosts_and_enrich'').hosts | length &gt; 0
}}'
        states:
          query_hosts_and_enrich: OK
      withItems: host in {{ result('query_hosts_and_enrich').hosts }}
      concurrency: 1
      input:
        script: |
          # paste the Step 6 script (apply_tags_to_hosts) here
```

6. Adapting It

- **Runtime inputs** — every knob (`DRY_RUN`, `MAX_HOSTS`, `APP_SCOPE`, `EXCLUDE_PRODUCTION`, `RESTART_ONEAGENT`, and the classic-only `ENVIRONMENT_URL` / `CREDENTIAL_ID`) is a workflow input, overridable per run in the Run dialog. No code edits to change scope or target.
- **Scope filters** — `APP_SCOPE` limits the run to one business app; `EXCLUDE_PRODUCTION` skips hosts whose enriched `environment` is `production`. Both are appended to the DQL at runtime, so task 1 accepts `APP_SCOPE` only as a plain identifier (`[a-z0-9._-]`) — a quote in the input cannot reach the query.
- **`sanitizeTagValue`** strips whitespace and `=` from the value and caps the whole `key=value` at 256 characters, the documented host-tag limit.
- **Batching** — the job API takes a list of `entities`. Sending several hosts that need the same tags in one job means fewer jobs and fewer 409 collisions.
- **Schedule it** with a cron trigger (WFLOW-02) so newly onboarded hosts get tagged automatically; skip-existing keeps repeat runs cheap.
- **Authentication** — platform authentication as the workflow actor (relative URL, `fleet-management:oneagents:write`) is the default; the classic `Api-Token` from the Credential Vault is kept only for classic or hybrid environments.

> **Sources:** OneAgent configuration via command-line interface (DT docs) — *"A property value must not contain = (unless used as a key-value delimiter) or whitespace characters. The maximum length is 256 characters, including the key-value delimiter."*; Remote configuration management (DT docs) — *"Remote configuration management does NOT work with: OneAgent deployed with Dynatrace Operator Application-only OneAgents OneAgents on Solaris"*.

7. Next Steps

- **WFLOW-02: Triggers** — add a schedule trigger to run this enrichment on a cadence.
- **WFLOW-07: Auto-Remediation** — the DRY_RUN / max-attempts / serialized-write guardrails here are the same safety patterns used for remediation tasks.
- **WFLOW-08: JavaScript & HTTP Actions** — the building blocks this LAB composes (SDK DQL, `fetch()`, retry, `result()` / `withItems`, Credential Vault).

- **WFLOW-09: Security, Governance & Monitoring** — service-user actors and the permissions a production workflow needs.
- **FAQ-02: Tagging — Sources, Standards & Strategy** — the tagging-source model and the `primary_tags.` prefix rule that this workflow writes against.

References

- OneAgent remote configuration management API — POST a configuration job (DT docs)
- Remote configuration management of OneAgents and ActiveGates (DT docs)
- Lookup data in Grail (DT docs)
- Build workflows (DT docs)
- Run JavaScript action (DT docs)
- OneAgent tag setup — primary tags / fields (DT docs)

> **Sources:** OneAgent remote configuration management API — POST a configuration job (DT docs) — documents the `hostTag` `set` operation, the `dt.cost.costcenter=<value>` form, and the `oneAgents.write` / `fleet-management:oneagents:write` scopes, Build workflows (DT docs), Lookup data in Grail (DT docs), OneAgent tag setup (DT docs).

Appendix — Bonus: Seed Sample Lookup Tables (Testing Only)

> **Not part of the main workflow.** In production the three CMDB lookup tables are uploaded from your **real CMDB** (Step 0). This appendix is only for trying the LAB on a tenant that has **no CMDB tables** — it creates three tiny sample tables so the enrichment resolves for one host. **Do not run it where the real tables already exist** (it would overwrite them).

It uses the Grail **Resource Store API** (`/platform/storage/resource-store/v1/files/tabular/lookup:upload`) — the reverse direction of this LAB (writing *to* a

lookup table instead of reading *from* one). The same API is how you'd build a real "sync CMDB → lookup table" job if your CMDB is reachable from a workflow.

Prerequisite: the workflow's actor needs `storage:files:write` to upload. Reading the tables back needs `storage:files:read` (the main LAB's actor has it); deleting a table needs `storage:files:delete`.

How to use: create a **separate** one-task workflow (don't add this to the tagging workflow), paste the code, **replace** `myhost-01` with the short hostname of a real host in your tenant (the part before the first dot), and run it once. Then verify with `load "/lookups/cmdb_server" | limit 10` and run the main workflow with `DRY_RUN = true`. The sample chain resolves to `application = sampleapp`, `environment = dev`.

```
const BASE = '/platform/storage/resource-store/v1/files/tabular';

const TABLES = [
  {
    filePath: '/lookups/cmdb_server', lookupField: 'name', displayName:
'cmdb_server',
    parsePattern: 'JSON{STRING:name, STRING:cmdb_ci_key,
STRING:dv_operational_status, STRING:environment, STRING:busapp_cmdb_ci_key,
STRING:apps_cmdb_ci_key}(flat=true)',
    rows: [{ name: 'myhost-01', cmdb_ci_key: 'SRV1', dv_operational_status:
'operational',
             environment: 'dev', busapp_cmdb_ci_key: 'BAPP1',
apps_cmdb_ci_key: 'APP1' }],
  },
  {
    filePath: '/lookups/cmdb_mapped_app_service', lookupField: 'cmdb_ci_key',
displayName: 'cmdb_mapped_app_service',
    parsePattern: 'JSON{STRING:cmdb_ci_key, STRING:environment,
STRING:managed_by_email, STRING:busapp_name}(flat=true)',
    rows: [{ cmdb_ci_key: 'APP1', environment: 'dev',
             managed_by_email: 'owner@example.com', busapp_name: 'Sample App'
}],
  },
  {
    filePath: '/lookups/cmdb_businessapp', lookupField: 'cmdb_ci_key',
displayName: 'cmdb_businessapp',
    parsePattern: 'JSON{STRING:cmdb_ci_key, STRING:short_name,
STRING:owned_by}(flat=true)',
    rows: [{ cmdb_ci_key: 'BAPP1', short_name: 'sampleapp', owned_by:
'platform-team' }],
  },
];

export default async function () {
  const out = {};
  for (const t of TABLES) {
    const jsonl = t.rows.map(r =&gt; JSON.stringify(r)).join('\n');

    // upload (multipart: request as a JSON string + content blob; NO headers)
    const form = new FormData();
    form.append('request', JSON.stringify({
      filePath: t.filePath,
      displayName: t.displayName,
      description: 'Sample CMDB data for testing host-tag enrichment',
      parsePattern: t.parsePattern,
      skippedRecords: 0,
      overwrite: true,
```

```
      lookupField: t.lookupField,
      timezone: 'UTC',
      locale: 'en_US',
    }));
    form.append('content', new Blob([jsonl], { type: 'text/plain' }),
'data.jsonl');

    const resp = await fetch(`${BASE}/lookup:upload`, { method: 'POST', body:
form });
    const body = await resp.text();
    console.log(`[upload] ${t.filePath} -&gt; HTTP ${resp.status} ::
${body.slice(0, 300)}`);
    out[t.filePath] = { status: resp.status, ok: resp.ok };
    if (!resp.ok) console.error(`Upload failed for ${t.filePath}: ${body}`);
  }
  console.log('Verify each: load "/lookups/cmdb_server" | limit 10');
  return out;
}
```

Upload mechanics (reusable for any lookup-table upload):

- `parsePattern: 'JSON{STRING:field, INT:field, …}(flat=true)'` produces **flat top-level columns** from JSONL. `(flat=true)` makes that explicit; when the pattern yields a single record-type field, the upload flattens it to the root by default anyway (`autoFlatten`).
- The `request` part is a plain **JSON string**; `content` is a `Blob`; **set no headers** — `fetch` builds the multipart boundary.
- The **relative URL** is authenticated by the platform as the workflow's actor — **no token needed**, the same mechanism task 2 of the main LAB uses.
- **Re-runnable:** `overwrite: true` replaces an existing table. To remove a table, POST `{"filePath": "/lookups/<name>"}` to `/platform/storage/resource-store/v1/files:delete` (needs `storage:files:delete`; deletion is irreversible).

> **Sources:** Lookup data in Grail (DT docs) — *"Delete the file from the Resource Store. POST/platform/storage/resource-store/v1/files:delete"*, *"To upload lookup data to Grail via REST API or to delete it, the policy bound to your user group must contain the following permissions: storage:files:write storage:files:delete"* and *"nested fields are extracted to the root level by default"*.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.